

AI-Based Interview Question Generator & Assessment Platform

Project Documentation - technical reference, architecture and user guide (derived from README.md)

Frontend	AI provider	Persistence	Runtime	License
Streamlit	Google Gemini (OpenAI alt. evaluator)	None - session state only	Python 3.10+	MIT

1. Overview

This Streamlit application uses Google Gemini to run a full technical interview: it **generates role-specific questions**, presents them one at a time, **scores each answer** (0-10) against a rubric, and compiles an **analytical performance report**. If the free-tier Gemini quota is exhausted (HTTP 429), it falls back to a built-in question generator and heuristic evaluator so the interview never stops. No database is used - all state lives in the Streamlit session. Typical users: candidates practising interviews, hiring managers screening applicants, and trainers assessing learners.

2. Key Features

Feature	Details
Question generation	Domains: Frontend, Backend, Full Stack, ML/AI, DevOps, Data Analysis, Custom. 5 experience levels (Fresher, 0-1, 1-3, 3-5, 5+ yrs), 3 difficulty tiers, categories: Technical, Conceptual, Scenario-Based, Problem Solving, Behavioral.
Gemini + fallback	Free-tier models: gemini-2.0-flash, 1.5-flash, 1.5-flash-8b, 2.0-flash-lite, 2.5-flash. On 429 / quota errors the engine switches automatically to dynamic templates and a heuristic evaluator.
Live interview	One question at a time, real-time answer capture, review tabs, and automated follow-up probing questions based on the candidate's answers.
Rubric evaluation	Strict 0-10 scoring. Output includes correct_points, missing_points, concise feedback and actionable improvement steps.
Analytics and reports	Percentage score, performance tier, topic-mastery breakdown, weak-topic detection with revision areas, and an exportable summary.
Zero-database design	Pure session-state memory; no SQL/NoSQL dependency, minimal setup.

3. Project Structure

File	Responsibility
app.py	Main Streamlit application and multi-page UI; manages session state and navigation.
services/question_generator.py	Gemini API caller and dynamic question generator; handles rubric scaling, invalid-JSON repair and quota fallback.
services/evaluator.py	Rubric-based answer evaluation; clamps scores to 0-max and produces feedback and follow-ups.
services/report_generator.py	Analytics engine: percentage, performance tier, topic breakdown, weak topics.
services/answer_evaluator.py	Alternative OpenAI-based evaluation service.
tests/test_qa_suite.py	12-point automated unit and integration test suite.
.env.example / requirements.txt / .gitignore	Environment template, Python dependencies, and Git ignore rules.

4. System Architecture

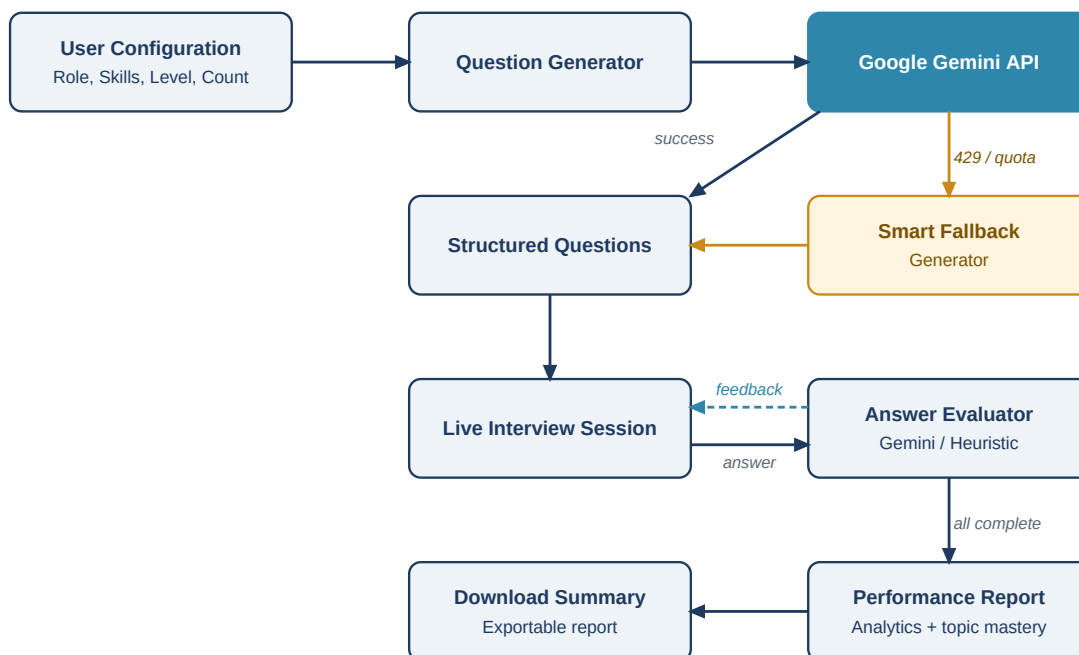


Figure 1 - Workflow. Dashed arrow: per-answer evaluation loop; amber path: quota fallback.

#	Stage	Description
1	Configuration	User picks role, skills, experience, interview type, difficulty and question count.
2	Generation	Generator requests structured questions (with rubrics) from Gemini; falls back to templates on 429/quota.
3	Live interview	Questions shown sequentially; candidate submits an answer to each.
4	Evaluation	Gemini or heuristic engine returns score, rubric feedback and a follow-up question.
5	Report and export	Scores aggregated into analytics and topic mastery; summary can be downloaded.

5. Installation and Setup

Prerequisites: Python 3.10+ and a free Google Gemini API key from Google AI Studio (aistudio.google.com).

```
# 1. Enter the project folder
cd HCL

# 2. (Optional) virtual environment
python -m venv .env
.\.env\Scripts\Activate.ps1      # Windows PowerShell
source .env/bin/activate         # Linux / macOS

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure the key: create .env (or copy .env.example)
GEMINI_API_KEY=your_actual_gemini_api_key_here

# 5. Launch, then open http://localhost:8501
streamlit run app.py
```

Environment variables: GEMINI_API_KEY (primary); GOOGLE_API_KEY and OPENAI_API_KEY are accepted as fallback names. Never commit .env to version control.

6. Configuration Options

Parameter	Options	Effect
Job Role	Frontend, Backend, Full Stack, Python, ML, DevOps, Data Analyst, Custom	Sets domain context.
Experience	Fresher, 0-1, 1-3, 3-5, 5+ yrs	Calibrates depth and difficulty.
Skills	Python, FastAPI, React, SQL, Docker, TypeScript, etc. + custom	Focuses on specific tools.
Interview Type	Technical, Behavioral, Coding, System Design, Mixed	Sets question structure and style.
Difficulty	Easy, Medium, Hard	Scales rubric strictness and scenario depth.
Question Count	5 to 15	Controls interview length.

7. Usage and Resiliency

Running an interview: (1) configure the session; (2) generate questions; (3) answer each question in turn; (4) review the 0-10 score, strengths, omissions, feedback and follow-up; (5) view the final report; (6) export the summary; (7) reset the session to start again.

Condition	System response
Valid Gemini response	Structured questions and evaluations are used directly.
Malformed JSON	Self-repair logic attempts to recover a valid structure.
HTTP 429 / quota exhausted	Automatic switch to built-in templates and the heuristic evaluator.
Empty or whitespace answer	Handled by a dedicated validation path.

8. Testing

Run: `python -m unittest tests/test_qa_suite.py -v`. The suite verifies:

#	Area	#	Area
1	Startup and constants integrity	7	Score clamping (0 to max)
2	Configuration dictionary generation	8	Follow-up question tracking
3	Input boundary and edge cases	9	Score and percentage accuracy
4	Question generation and rubric scaling	10	Final report metrics and topic breakdown
5	Invalid JSON handling and repair	11	State reset and session clearing
6	Empty / whitespace answers	12	Zero-database constraint

9. Limitations and README Review Notes

- **Limitations:** no persistence (export the report before closing); free-tier quotas trigger the less nuanced fallback; heuristic scoring is not equivalent to LLM rubric scoring; verify the listed Gemini models are still offered; no authentication or candidate history.
- **Personal path:** the README's setup step contains a machine-specific Windows path - replace it with a generic one.
- **Inconsistent lists:** Interview Type options differ from the question categories, and the Job Role lists differ slightly between sections (Python appears only in the config table).
- **Unclear default evaluator:** both `evaluator.py` (Gemini) and `answer_evaluator.py` (OpenAI) exist, but the README does not say which is used.
- **Missing items:** no LICENSE file in the structure despite the MIT claim; no screenshots, sample report or deployment instructions.